

第 47 讲：LayerNorm 与 RMSNorm

为什么 Transformer / LLM 更喜欢对每个 Token 的 Hidden Dimension 做归一化

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

前两讲已经回答了两个问题：第 45 讲解释“为什么深层网络需要 Normalization”，第 46 讲解释“BatchNorm 怎样依赖 Mini-Batch 统计量”。本讲进入 Transformer / LLM 最直接相关的归一化方式：**LayerNorm** 与 **RMSNorm**。

本讲的最低目标不是背公式，而是以后看到 (B, T, D) 的 Hidden States 时，能立即判断：**到底沿哪个 Axis 计算统计量？同一个 Token 会不会受别的样本影响？train/eval 会不会换统计量？**

0. 学完以后，你应该能做到什么？

- 真正解释 LayerNorm 的输入、输出、统计轴、mean、variance、 γ 、 β 。
- 用一个四维小向量手算一次 LayerNorm。
- 真正解释 RMSNorm 与 LayerNorm 的相同点和不同点。
- 看到 $X: (B, T, D)$ 时，知道 `nn.LayerNorm(D)` 通常对每个 Token 独立沿 D 归一化。
- 解释为什么 LayerNorm / RMSNorm 不依赖 Batch 同伴，也没有 BatchNorm 那样的 running mean / variance。
- 理解 Pre-Norm / Post-Norm 描述的是 Norm 在 Residual Block 中的位置，而不是另一套归一化公式。
- 看懂 Transformer / LLM 源码中的 LayerNorm / RMSNorm，并判断常见 Shape 是否合理。

【科研连接】

你以前在 LLM 课程里已经“见过”Transformer Block、LayerNorm、RMSNorm、Pre-Norm。本讲不把它们当作黑盒模块，而是从 Tensor、Axis、均值、方差、RMS 和可学习参数重新理解。

1 先把 BatchNorm、LayerNorm、RMSNorm 放在同一张地图上

假设 Transformer 的 Hidden States:

$$X \in \mathbb{R}^{B \times T \times D}$$

其中:

- B : Batch Size, 一批有多少条序列;
- T : Sequence Length, 每条序列有多少个 Token;
- D : Hidden Dimension, 每个 Token 用多少个数字表示。

【原理】

Normalization 的核心并不是“把数字变小”，而是：先决定一组哪些元素共同计算统计量，再用这些统计量重新标定这一组数值。
因此读任何 Normalization，第一问应该是：统计量沿哪些 Axis 计算？

假设 Hidden States 的 Shape = (B, T, D)

BatchNorm (概念类比)	同一 feature/channel 跨样本 (及空间/位置) 统计
LayerNorm	每个 Token 自己沿最后的 Hidden Dimension D 统计 mean + variance
RMSNorm	每个 Token 自己沿 D 统计 RMS ; 不先减均值

关键问题：Normalization 到底沿哪些 Axis 计算统计量？

方法	典型统计单位	是否依赖 Batch 同伴	是否有 running statistics
BatchNorm	同一个 Channel 跨 Batch (以及空间/位置) 统计	是	通常有
LayerNorm	每个样本/Token 自己沿 Hidden Dimension 统计	否	没有
RMSNorm	每个样本/Token 自己沿 Hidden Dimension 统计 RMS	否	没有

【通俗解释】

把一个 Token 想成一个学生的“D 维成绩单”。LayerNorm / RMSNorm 是“每个学生拿自己的各科成绩做内部标定”；BatchNorm 更像“每一科拿全班学生的这一科成绩做统计”。所以 BatchNorm 会受同班同学影响，而 LayerNorm / RMSNorm 不会。

2 LayerNorm：到底做了什么？

设一个 Token 的 Hidden Vector 为：

$$x = [x_1, x_2, \dots, x_D]$$

LayerNorm 最常见的第一步，是在这个 Token 自己的 D 个 Hidden Features 上计算均值：

$$\mu = \frac{1}{D} \sum_{i=1}^D x_i$$

再计算总体方差：

$$\sigma^2 = \frac{1}{D} \sum_{i=1}^D (x_i - \mu)^2$$

然后标准化：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

最后加入可学习的缩放与平移：

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

【原理】

LayerNorm 有两个逻辑阶段：

1. **Normalize**：减均值、除以标准差，让当前 Token 内部的数值尺度更可控；
2. **Affine restore**：通过可学习的 γ 和 β ，让网络有能力重新选择合适的尺度和偏移，而不是被强制永远保持“均值 0、方差 1”。

为什么需要 ϵ ？

如果一个 Token 的 D 个元素几乎完全一样，方差可能接近 0。直接除以 $\sqrt{\sigma^2}$ 会带来除零或数值不稳定，因此使用：

$$\sqrt{\sigma^2 + \epsilon}$$

其中 ϵ 是一个很小的正数。

【常见误区 / 风险】

不要把 ϵ 理解成“正则化强度”。它首先是数值稳定项。不同实现、不同 dtype 下，合适的 ϵ 可能不同；做严格复现时应把它当成模型配置的一部分记录。

3 LayerNorm: 用四个数字从头手算一次

【具体数字例子】

取：

$$x = [1, 2, 3, 4]$$

第一步，均值：

$$\mu = \frac{1 + 2 + 3 + 4}{4} = 2.5$$

第二步，各元素减均值：

$$[-1.5, -0.5, 0.5, 1.5]$$

第三步，平方：

$$[2.25, 0.25, 0.25, 2.25]$$

第四步，方差：

$$\sigma^2 = \frac{2.25 + 0.25 + 0.25 + 2.25}{4} = 1.25$$

忽略很小的 ϵ 时：

$$\sigma \approx 1.1180$$

因此：

$$\hat{x} \approx [-1.3416, -0.4472, 0.4472, 1.3416]$$

实验中 PyTorch 得到完全相同的结果，最大绝对误差为 0。LayerNorm 后：

$$mean \approx 0, \quad variance \approx 1$$

【代码实验】

```
x = torch.tensor([1., 2., 3., 4.])
ln = nn.LayerNorm(4, elementwise_affine=False)
y = ln(x)
```

如果开启默认的 `elementwise_affine=True`，则 LayerNorm 还会保存：

$$\gamma : (D,), \quad \beta : (D,)$$

例如 $D = 4$ 时共有 8 个可学习参数。

【常见误区 / 风险】

“LayerNorm 后均值一定是 0、方差一定是 1” 只严格描述**标准化后的** $\hat{x}$ 。一旦再经过可学习的 γ, β ，最终输出 y 的均值和方差可以被重新改变。

4 RMSNorm：为什么不减均值也能工作？

RMS 是 Root Mean Square（均方根）：

$$RMS(x) = \sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2 + \epsilon}$$

RMSNorm 的核心形式是：

$$\hat{x}_i = \frac{x_i}{RMS(x)}$$

再乘可学习 Scale：

$$y_i = \gamma_i \hat{x}_i$$

【原理】

RMSNorm 与 LayerNorm 的共同目标都是控制一个 Token 内部 Hidden Vector 的尺度，但 RMSNorm 去掉了“减均值”这一步。

因此：

- LayerNorm 控制**中心位置 + 尺度**；
- RMSNorm 主要控制**均方根尺度**。

对同一个：

$$x = [1, 2, 3, 4]$$

先计算：

$$RMS = \sqrt{\frac{1^2 + 2^2 + 3^2 + 4^2}{4}} = \sqrt{7.5} \approx 2.7386$$

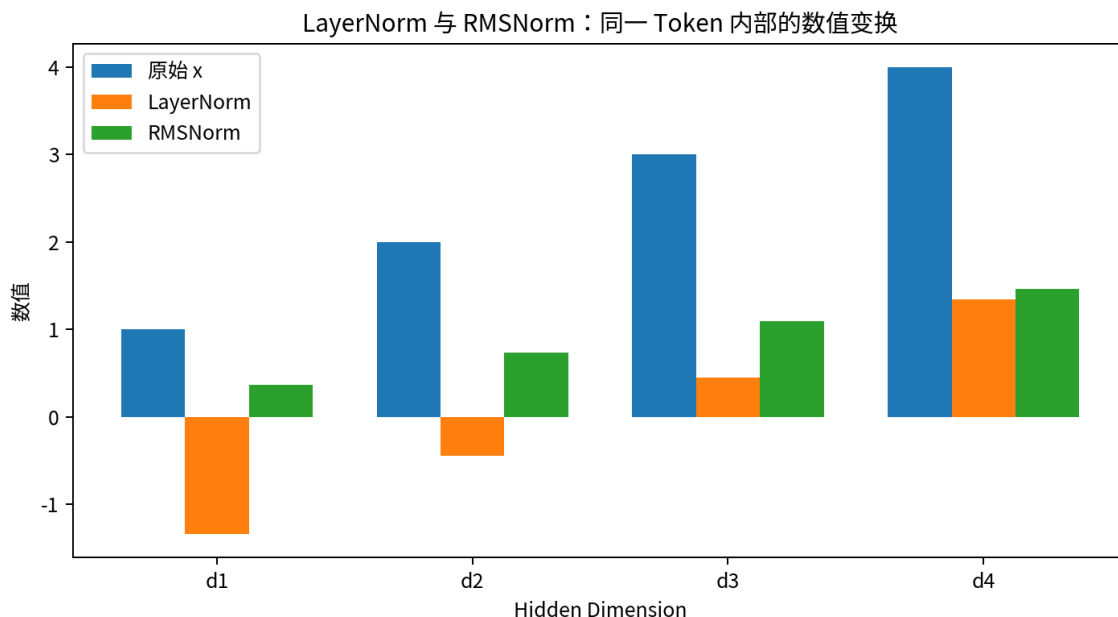
因此：

$$RMSNorm(x) \approx [0.3651, 0.7303, 1.0954, 1.4606]$$

它的 RMS 约为 1，但均值：

$$mean \approx 0.9129$$

并不要求为 0。



【科研连接】

现代 LLM 中常见 RMSNorm。一个重要直觉是：如果任务中“去均值”不是必要条件，那么省去 mean-centering 可以让归一化更简洁。但不能据此推导“RMSNorm 在所有模型、所有任务上一定优于 LayerNorm”；实际效果依赖架构和训练设置。

5 LayerNorm 与 RMSNorm 最关键的差异：是否做 mean-centering

考虑给所有 Hidden Features 同时加一个常数 c ：

$$x' = x + c\mathbf{1}$$

LayerNorm 中均值也同步增加 c ：

$$\mu' = \mu + c$$

所以：

$$x' - \mu' = x - \mu$$

因此在相同 γ, β 和忽略数值误差时，LayerNorm 对这种“整体平移”具有不变性。

RMSNorm 不减均值，所以整体加常数会改变：

$$RMS(x + c)$$

以及归一化结果。

实验中：

$$x = [1, 2, 3, 4]$$

改成：

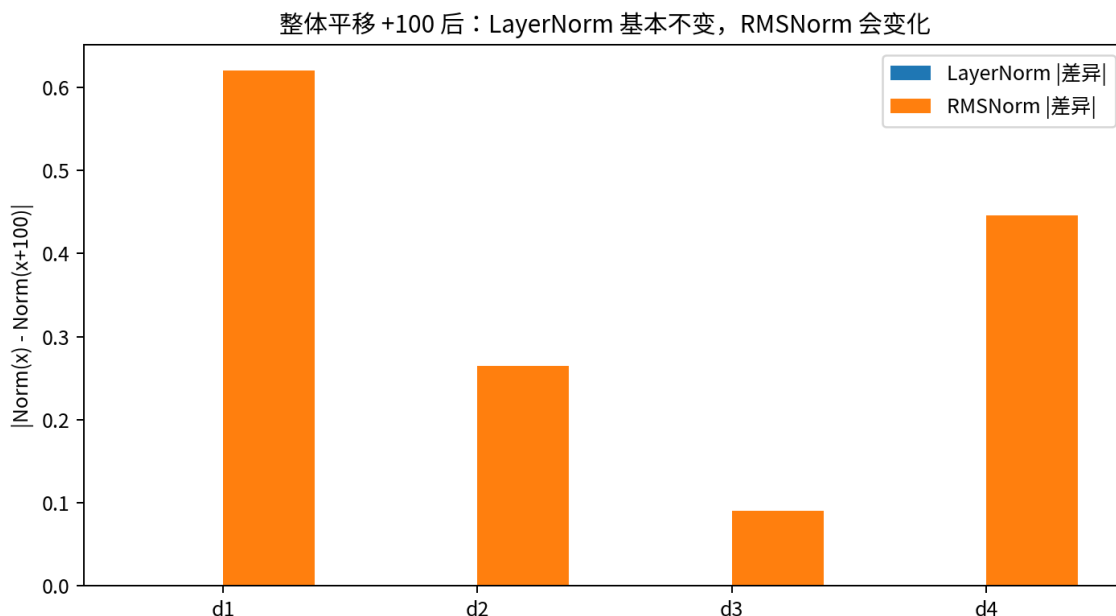
$$x + 100 = [101, 102, 103, 104]$$

LayerNorm 输出最大差异：

$$0$$

而 RMSNorm 最大差异约：

$$0.6202$$



【常见误区 / 风险】

不要把“LayerNorm 对统一平移不变”理解成“LayerNorm 会忽略所有绝对数值信息”。它只是对沿被归一化维度的**统一加常数**有这一性质；实际网络中还有 γ, β 、Residual、Linear、Attention 等大量操作。

6 为什么 LayerNorm / RMSNorm 特别适合 (B,T,D) 的 Transformer Hidden States?

设：

$$X.shape = (B, T, D)$$

调用：

```
ln = nn.LayerNorm(D)
y = ln(X)
```

通常意味着：对每一个 (b, t) 位置，拿自己的：

$$X[b, t, :]$$

这 D 个数字计算统计量。

所以不同 Batch、不同 Token 之间不会共同计算均值/方差。

【具体数字例子】

如果：

$$X.shape = (2, 3, 4)$$

那么共有：

$$2 \times 3 = 6$$

个 Token Vector。LayerNorm 会分别对 6 个长度为 4 的向量做归一化，而不是把 24 个数字混在一起算一个均值。

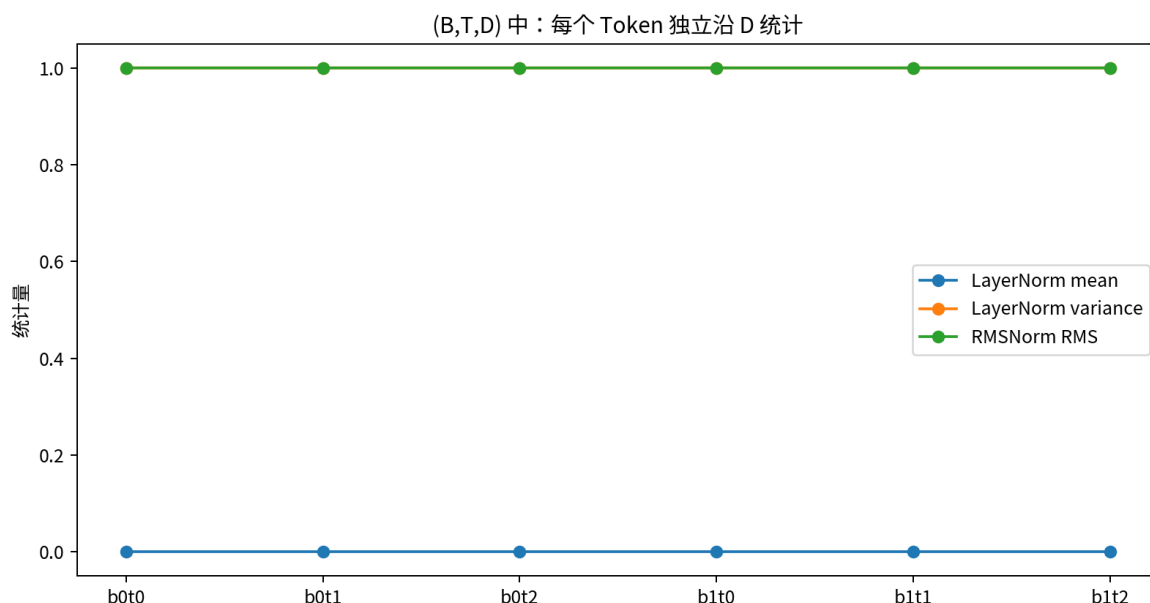
实验中 LayerNorm 对每个 Token 的：

$$mean \approx 0, \quad variance \approx 1$$

而 RMSNorm 对每个 Token 的：

$$RMS \approx 1$$

但 mean 可以是正、负或接近 0。



【通俗解释】

对 LLM 来说，一个 Token 的 Hidden Vector 就像这个 Token 当前“脑内状态”的 D 维表示。LayerNorm / RMSNorm 让每个 Token 可以独立整理自己的内部数值尺度，而不要求“必须凑够一个稳定 Batch”才能知道应该怎么归一化。

7 Batch Dependency: 为什么同一个样本在 BatchNorm 里会变, 在 LayerNorm 里不会?

本讲实验把完全相同的 target:

$[1, 2, 3, 4]$

放进两个不同的 Batch。

在 Batch A 中, 它的 BatchNorm 输出是:

$[-0.4472, 0.2294, 0.8165, 1.1832]$

换到 Batch B 后变为:

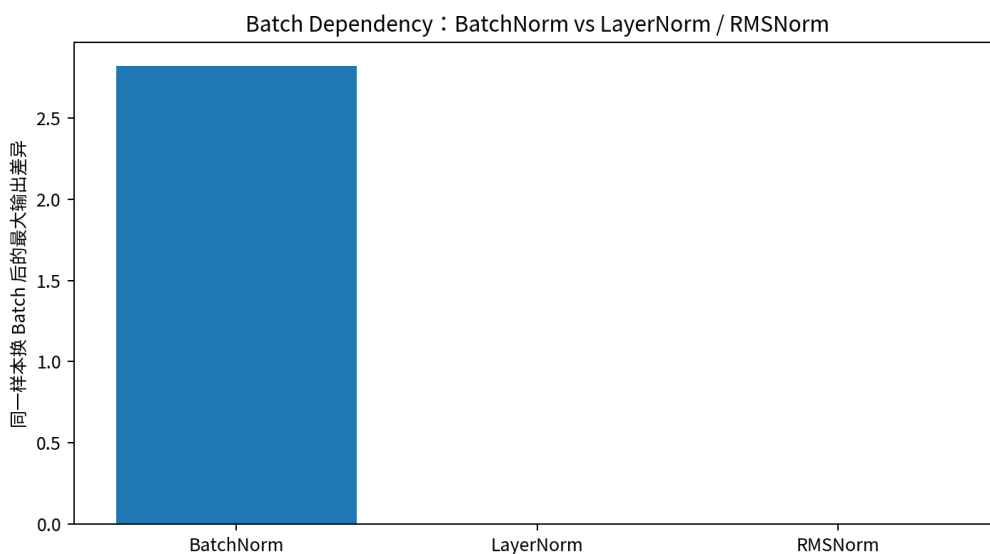
$[-1.6419, -1.6405, -1.6391, -1.6377]$

最大差异约:

2.8209

而 LayerNorm / RMSNorm 的最大差异都是:

0



【原理】

原因不是“LayerNorm 更稳定”这么简单, 而是**统计依赖结构不同**:

- BatchNorm 的当前输出依赖一组样本共同形成的 statistics;
- LayerNorm / RMSNorm 的当前输出只依赖当前 Token 自己的 Hidden Vector (以及可学习参数)。

这对 Variable Sequence Length、Autoregressive Generation、小 Batch、单样本推理尤其自然: 模型不需要等待别的样本来组成稳定的 Batch Statistics。

8 `train()` / `eval()`: LayerNorm 和 RMSNorm 为什么没有 BatchNorm 那种切换?

BatchNorm 在训练阶段和评估阶段统计来源不同：训练时看当前 Batch，评估时通常看 running statistics。

LayerNorm / RMSNorm 则没有这套 running mean / running variance。对同一个输入，单独看 Norm 模块本身：

`train()` 和 `eval()`

数学输出相同。

本讲实验：

$$\max |LayerNorm_{train} - LayerNorm_{eval}| = 0$$

$$\max |RMSNorm_{train} - RMSNorm_{eval}| = 0$$

【常见误区 / 风险】

这不等于“整个 Transformer 在 train/eval 下一定输出相同”。Transformer Block 里还可能存在 Dropout。`model.eval()` 会关闭 Dropout，因此整个模型仍可能变化。这里说的是：**LayerNorm / RMSNorm 自身没有 BatchNorm 那种 statistics mode switching。**

`state_dict` 里保存什么?

LayerNorm 一般保存可学习的：

$$\gamma : (D,), \quad \beta : (D,)$$

PyTorch 的 `nn.RMSNorm(D)` 默认保存：

$$\gamma : (D,)$$

而没有 bias 参数，也没有 running mean / variance。

【科研连接】

做模型 checkpoint 对比时，如果看到 LayerNorm/RMSNorm 的状态量远少于 BatchNorm，不是“保存漏了”，而是机制本来就不同。

9 LayerNorm / RMSNorm 的 Backward：为什么仍然不是简单逐元素缩放？

虽然 LayerNorm 的 Forward 可以写成：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

但 μ 和 σ^2 都由同一组 D 个元素共同计算。

所以一个输入 x_i 改变时，它不仅影响自己的分子，还会影响：

$$\mu$$

和：

$$\sigma^2$$

进而间接影响同一个 Token 内的其他元素。

【原理】

因此 Normalization 的 Backward 具有“组内耦合”：同一个归一化组里的元素不是完全互不相关地传 Gradient。

RMSNorm 虽然没有 mean-centering，但 RMS：

$$\sqrt{\frac{1}{D} \sum x_i^2 + \epsilon}$$

同样由所有 D 个元素共同决定，所以也存在组内耦合。

当前阶段不要求你手推完整 Jacobian，但必须保留两个底层直觉：

1. Norm 是一个可微计算图，Gradient 会穿过 mean / variance / RMS；
2. “沿 D 统计”不仅决定 Forward，也决定哪些元素在 Backward 中彼此耦合。

【科研连接】

以后读高效 Transformer Kernel、Fused LayerNorm / RMSNorm、数值稳定性论文时，这个计算结构会再次出现。但当前课程不需要进入 CUDA Kernel 级别。

10 Pre-Norm 与 Post-Norm: Norm 放在 Residual 的哪一边?

一个抽象的子层 $F(\cdot)$ ，可以是 Attention，也可以是 FFN。

Pre-Norm

$$Y = X + F(Norm(X))$$

先 Normalize，再进入子层，最后与 Residual 相加。

Post-Norm

$$Y = Norm(X + F(X))$$

先做子层与 Residual 相加，再 Normalize。

【通俗解释】

“Pre / Post”说的是 Norm 相对于 Residual / SubLayer 的位置；“LayerNorm / RMSNorm”说的是 Norm 内部怎样计算统计量。它们是两个不同维度的问题。

【代码实验】

```
# Pre-Norm
x = x + sublayer(norm(x))

# Post-Norm
x = norm(x + sublayer(x))
```

本讲实验只验证两者 Shape 都保持：

$$(B, T, D) \rightarrow (B, T, D)$$

不会据此宣称某一种在所有模型上一定更好。

【科研连接】

以后论文如果比较 Pre-Norm / Post-Norm，不要只看最终 Accuracy。还应看深度、初始化、Learning Rate、训练稳定性、Gradient Norm、是否使用额外 Scaling 等控制变量。

11 为什么现代 Transformer / LLM 经常使用 LayerNorm 或 RMSNorm?

把前面的机制连起来，可以得到几个结构层面的原因：

1. **不依赖 Batch Statistics**：单个 Token / 单条序列也可以稳定执行；
2. **适合 (B,T,D) 表示**：自然沿每个 Token 的 Hidden Dimension D 工作；
3. **train/eval 统计逻辑一致**：没有 running mean / variance 的切换；
4. **与 Residual Block 容易组合**：Pre-Norm / Post-Norm 都可以自然插入；
5. **RMSNorm 结构更简洁**：不做 mean-centering，只控制 RMS 尺度。

【常见误区 / 风险】

“现代 LLM 常用 RMSNorm” 不等于 “LayerNorm 已经过时”，也不等于 “RMSNorm 在任何实验中一定更好”。架构选择是经验、稳定性、效率和历史演化共同作用的结果。做 Ablation 时应控制其余配置。

参数量示例

若 $D = 4096$ ：

- LayerNorm 若有 γ, β ：约 8192 个参数；
- RMSNorm 若只有 γ ：约 4096 个参数。

相对于数十亿参数的 LLM，这不是主要参数开销，但计算路径和数值行为仍然重要。

12 常见误区与科研诊断

误区 1: LayerNorm 就是“把整个 Tensor 变成均值 0 方差 1”

错误。LayerNorm(D) 在 (B, T, D) 上通常是**每个** (b, t) **Token 自己**沿最后的 D 维统计。

误区 2: RMSNorm 后均值也应该是 0

错误。RMSNorm 不减均值，只约束均方根尺度。实验中 $[1, 2, 3, 4]$ 经 RMSNorm 后均值约为 0.9129。

误区 3: model.eval() 会关闭 LayerNorm / RMSNorm

错误。eval 并不会“关闭”它们；只是某些模块如 Dropout、BatchNorm 会切换行为。LN/RM-SNorm 自身仍继续计算。

误区 4: Normalization 一定提高 Generalization

Normalization 首先服务于数值尺度和 Optimization。它可能间接影响泛化，但不能机械推断“用了 Norm 就一定泛化更好”。

训练出现异常时可以记录什么？

- Norm 前后每层 Activation mean / std / RMS;
- γ 的分布与 Gradient Norm;
- 是否出现 NaN / Inf;
- Residual Stream 的 Norm 是否随深度系统性增长;
- 不同 ϵ 、Pre/Post-Norm、LayerNorm/RMSNorm 的公平 Ablation。

【科研连接】

科研中“Normalization 改进有效”通常至少需要回答：改变的是统计轴、center/scale 机制、参数化、放置位置，还是纯粹的工程 Kernel？不要把多个改动一起打包后只归因于一个名词。

13 完整代码最小闭环：从 Hidden States 到 Transformer Norm

【代码实验】

```
B, T, D = 2, 3, 4
x = torch.randn(B, T, D)

ln = nn.LayerNorm(D)
rms = nn.RMSNorm(D)

x_ln = ln(x)      # (B,T,D) -> (B,T,D)
x_rms = rms(x)    # (B,T,D) -> (B,T,D)

# Pre-Norm residual block
x = x + sublayer(rms(x))
```

看到这段代码时，应该逐行翻译成：

1. 当前有 $B \times T$ 个 Token Vector；
2. 每个 Vector 长度 D ；
3. Norm 对每个 Vector 自己沿 D 统计；
4. Shape 不变；
5. RMSNorm 不做 mean-centering；
6. Pre-Norm 是先 Norm 再进 sublayer，最终加 Residual。

本讲总装图

$$(B, T, D) \rightarrow \begin{cases} LayerNorm : mean + variance \\ RMSNorm : RMS \end{cases} \rightarrow (B, T, D)$$

最低掌握标准

学完本讲后，看到 `nn.LayerNorm(4096)` 或 `nn.RMSNorm(4096)`，你应该能立即解释：4096 是 Hidden Dimension；每个 Token 自己沿这 4096 个值计算统计量；Batch Size 和 Sequence Length 不参与共同统计；输出 Shape 不变；LayerNorm 会中心化，RMSNorm 不中心化。

5 分钟口述自检

不看讲义，尝试口头回答：

1. 对 $X.shape = (8, 512, 4096)$ ，`LayerNorm(4096)` 到底在哪些元素上计算 mean / variance？
2. 为什么同一个 Token 换 Batch 同伴后，`LayerNorm` 输出不变，而 `BatchNorm` 可能变化？
3. `LayerNorm` 和 `RMSNorm` 的核心数学差异是什么？
4. 为什么 `RMSNorm` 输出均值不要求为 0？
5. `model.eval()` 会不会让 `LayerNorm` 停止工作？为什么？
6. `Pre-Norm` 和 `RMSNorm` 是同一个维度的概念吗？
7. 如果一篇论文说“`RMSNorm` 更好”，你至少希望它控制哪些变量？

【科研连接】

如果这 7 个问题能用自己的话说清楚，而不是复述定义，本讲就达到了“真正理解”的最低标准。

下一讲预告

第 48 讲进入 CNN 基础。CNN 不会学得很深，目标是理解另一类经典神经网络怎样利用局部连接、Kernel、Feature Map 和参数共享，从而进一步建立“深度学习模型为什么要根据数据结构设计计算结构”的整体直觉。